

Copilot instructions for this repository

Purpose: orient Copilot/Copilot CLI sessions to the repo's layout, run-time commands, and important conventions so suggestions and automated edits target the right files and behaviors.

Quick run / dev commands

- Start the local web server (serves static site + small REST endpoints):

From repo root:

```
python src/public_html/server.py  
# then open: http://localhost:8080/index.html
```

- Alternate (project's README uses uv helper):

```
# from the environment the author used  
uv sync  
uv run python src/public_html/server.py
```

- Tests / lint: none detected in this repository (no package.json, pytest/tox/flake8 configs). If adding tests, place them under test/ or use pytest-style test_*.py files.
-

High-level architecture

- This repository is primarily a static/public HTML site under src/public_html/ with a small Python HTTP server at src/public_html/server.py that:
 - Serves static files and directory listings
 - Injects a shared dark-mode "shell" (CSS/JS) into served HTML documents
 - Exposes simple REST endpoints for persistence and health:
 - * GET/POST /api/text/<key> — read/write saved/<key>.txt
 - * /api/ping — heartbeat used to keep the server alive
 - * /api/shutdown — ignored by default (server lifetime is heartbeat-controlled)
 - Provides legacy URL rewrites/aliases so older file:// URLs map to repository assets
- Persistent data: src/public_html/saved/ (server writes <key>.txt files there).

- Entry/start file: 0101.html (see nbproject/project.properties for start.file).
 - Default port: 8080 (variable PORT in server.py).
-

Key repository conventions (important for automated edits)

- server.py is authoritative for how HTML is delivered and transformed. Any automation that modifies HTML/CSS/JS served to users should account for:
 - HTML normalization and injection performed by `_normalize_html_document` and `_inject_shell_assets` in `server.py`.
 - Legacy URL rewriting via `_rewrite_legacy_file_urls` and the `LEGACY_ASSET_ALIASES` map.
 - API key form: keys are validated with `KEY_RE` in `server.py` — only lowercase letters, digits and hyphens, starting with an alphanumeric, max 64 chars. Saved files are named `<key>.txt` in `saved/`.
 - Image fallback: server will serve `missing-asset.svg` when requested image file variants are missing (see `IMAGE_FALLBACK_EXTENSIONS`).
 - NetBeans metadata: `nbproject/` contains project settings (source folders, site root). `nbproject/project.properties` is a good source for canonical paths used by the project.
 - Local dev helper: the repo references a `uv` helper (used in README) for syncing and running. Copilot-run commands should prefer simple `python src/public_html/server.py` unless the user's environment provides `uv`.
-

Files to consult when making changes

- `src/public_html/server.py` — core HTTP server and HTML transformation logic (read before changing HTML-serving behavior).
- `src/public_html/` — static assets, `saved/` folder for persisted text.
- `nbproject/project.properties` — project layout metadata (start.file, web context root).
- `README.md` — local startup notes and environment hints.

If adding automated tests or linters, add standard config files (pytest, tox, package.json, or pyproject.toml) and update this doc.

Suggested order of expansion (prioritized)

1. Add unit tests (pytest)
 - Rationale: fast feedback loop, lowest friction. Start with small unit tests for server helpers in `src/public_html/server.py` (e.g., URL rewrite, key validation, HTML normalization).
 - How a Copilot session should run a single test once tests are added:

```
# run a single test file
pytest test/test_server.py
# run a single test by node
pytest test/test_server.py::test_normalize_html
```
2. Add continuous integration (GitHub Actions)
 - Rationale: ensures tests run on PRs and main branch. Create a minimal workflow that installs Python, installs deps, runs pytest, and optionally runs linters.
 - Files: `.github/workflows/ci.yaml`.
3. Add linters & formatters (black, isort, flake8)
 - Rationale: consistent style and automated fixes in PRs. Add config files (`pyproject.toml`, `.flake8`, `setup.cfg`) and include lint steps in CI.
4. Add type-checking (mypy) and stricter static checks
 - Rationale: improves safety for refactors. Add type hints incrementally.
5. Add end-to-end/browser tests (Playwright or Selenium)
 - Rationale: validate the served site and the injected shell. Run headless browser tests against `localhost:8080` in CI or via a test job.
6. Add MCP server integrations and automation
 - Rationale: enable hosted agents for running E2E suites, batch evals, or prompt-based test generation.

MCP servers to consider (recommended for this repo)

- Playwright MCP / Headless Chromium (recommended)

- Use when adding E2E/browser tests that exercise the server's HTML injection and legacy URL rewrites.
- What Copilot should do: add a playwright test job in CI, or configure an MCP server named like playwright (server id: playwright) that can run Playwright scripts against `http://localhost:8080`.
- Python / pytest runner (useful)
 - A simple Python test-runner MCP can run pytest, capture outputs, and be used for batch test runs and trace collection.
- Browser screenshot / visual regression server (optional)
 - For visual diffs of the injected shell across changes.

If the user wants, Copilot can scaffold example GitHub Actions workflows and sample MCP server config files for Playwright and pytest.

How Copilot should approach changes

- Read `src/public_html/server.py` before modifying how HTML/CSS/JS is served; the server performs normalization, injection, and legacy URL rewrites.
 - When adding tests or CI, prefer small commits that add test scaffolding first (tests-only commit), then add CI and lints in follow-ups.
 - For E2E tests, start with one deterministic scenario (open `index.html`, assert shell injection and a known element) before expanding coverage.
-

If you'd like, scaffold the following next: a minimal pytest test for `server.py`, a GitHub Actions `ci.yaml`, or a Playwright example and MCP server configuration. Tell me which to scaffold first and it will be created.

Makefile helper

- A Makefile target `download-artifacts` is available to fetch workflow artifacts using the included script. Example:

```
make download-artifacts OWNER=your-org REPO=your-repo RUN_ID=123456 ARTIFACT_NAME=
```

This runs `scripts/download_artifacts.py` and extracts artifacts into the specified `OUT_DIR`.

Auto-download helper

A small helper script is provided at `scripts/download_artifacts.py` to download and extract workflow artifacts for a given workflow run. It uses the GitHub Actions API and requires a GitHub token with repo access.

Usage example (local):

```
export GITHUB_TOKEN=ghp_... # a personal access token or GITHUB_TOKEN in CI
python scripts/download_artifacts.py \
  --owner your-org-or-user \
  --repo your-repo \
  --run-id 123456789 \
  --artifact-name e2e-artifacts \
  --out-dir /tmp/e2e-artifacts
```

Notes for Copilot sessions: - The PR comment posted by CI links to the artifact; use that run id when invoking the helper. - If running in CI or scripts, the `GITHUB_TOKEN` environment variable can be used instead of passing `--token`. - The script downloads the artifact ZIP and extracts it to `--out-dir`; filenames inside match what the E2E tests write (screenshots and HTML files in `e2e/artifacts`).

CI artifacts & PR comments

- The E2E workflow uploads failing-run artifacts (`e2e/artifacts/*`) and retains them for 7 days.
 - When an E2E job fails on a pull request, the workflow posts a comment on the PR with a link to the uploaded artifacts so reviewers can download screenshots and page HTML for debugging.
 - Copilot sessions may fetch these artifacts by following the link in the PR comment; when suggesting fixes, reference artifact filenames (screenshots and HTML) to help reviewers reproduce failures.
-

Cleanup helper

- A small script `scripts/clean.sh` is provided to remove local test artifacts and the virtual environment created during local runs.

Usage:

```
# remove .venv, test artifacts, and server log
./scripts/clean.sh
```

- The repository's `.gitignore` now ignores `.venv/` and `e2e/artifacts/` so generated files are not committed.